

159.735 2025/S2—Assignment 1

Parallel random numbers and Monte Carlo integration

Monte Carlo methods using random numbers are used extensively in numerical computation and analysis. Here, we implement a Monte Carlo method to evaluate the gamma function. For a complex value, z , the gamma function is defined as:

$$\Gamma(z) = \int_0^\infty t^{z-1} e^{-t} dt$$

For this exercise we evaluate the gamma function for real positive values x . In this case the above improper integral can now be written as an integral with finite limits:

$$\Gamma(x) = \int_0^1 \left(\log \frac{1}{t} \right)^{x-1} dt$$

where the log function is the “natural” logarithm (log to the base e).

In the method of Monte Carlo integration, the integral can be estimated by sampling random values between two limits and accumulating the function (integrand) evaluations over this sample, i.e.

$$I(N) = (t_2 - t_1) \frac{1}{N} \sum_{n=0}^{N-1} f(t_n)$$

where t_n is a random number between t_1 and t_2 and N is the total of the sample. The larger the sample, the more accurate the integral estimator.

Note: the gamma function has the special property for integer arguments n :

$$\Gamma(n) = (n-1)!$$

e.g.

$$\Gamma(5) = 4! = 4 \times 3 \times 2 \times 1 = 24$$

This could come in useful for checking the accuracy of the Monte Carlo integration estimator.

Implementation

For this assignment, write an MPI program to implement the parallel computation of the gamma function for a given positive and real x , using Monte Carlo integration.

You will need to decide upon a strategy for generating random numbers and to ensure that all processes get their random numbers from the same pool. Doing this in a parallel computation environment is a key purpose of this exercise. You should use the “leapfrog” method together with a linear congruential operator as described in the lectures.

For a given value of x , you are asked to parallelize the Monte Carlo integration procedure. To compute the integrand, you can use appropriate C/C++ library functions.

Once your program is working, test it against Amdahl’s Law by carrying appropriate benchmarking runs.

Submission

Submit your C or C++ source code together with a brief report which addresses the following.

- Briefly comment on the accuracy of your implementation.
- Give the results of your performance testing of your implementation of the parallel Γ calculation. Present your results in such a way that best demonstrates Amdahl’s Law. Are the parallel speed-ups what you would expect?
- Measure the interprocess communication times. Provide an *estimate* of the amount of time your program will require if one implemented a method where only the master generates the random numbers and sends each to its appropriate slave.

This assignment is worth 20% of your final grade